



DAG, JOBS, STAGES & TASKS

How Spark Turns One Action Into Millions of Parallel Computations

Job

Stage

Task

Shuffle

Interview Ready in 6 Slides

Swipe to start learning →

J ONE ACTION → ONE JOB

The moment an Action runs, the **DAGScheduler** walks the logical plan backwards from the final RDD/DataFrame and builds a **Directed Acyclic Graph** of every step needed to produce the result.

The Chain of Command

1

An Action runs — **.show(), .count(), .collect()** — triggers exactly **one Job**



2

DAGScheduler walks the plan backwards, building the full dependency graph



3

Every **wide dependency** (shuffle) becomes a new Stage boundary



4

Each Stage is split into **Tasks** — one per partition

Interview Tip: 1 Action = 1 Job. But one Job can contain many Stages, and each Stage can contain hundreds of Tasks.

S STAGE BOUNDARIES

NARROW DEPENDENCIES (NO SHUFFLE — SAME STAGE)

`.map()`

`.filter()`

`.union()`

`.withColumn()`

WIDE DEPENDENCIES (SHUFFLE — NEW STAGE)

`.groupBy()`

`.join()`

`.distinct()`

`.repartition()`

Why the Boundary Exists

Spark only opens a new Stage when data has to **move across partitions** — a shuffle. Everything before that boundary can run in one uninterrupted pass, called a Stage.

Interview Tip: Count the shuffles in your execution plan and you've counted your Stages, minus one.

T TASK — THE UNIT OF PARALLELISM

A **Task** is the smallest unit of work Spark schedules — it runs one Stage's logic against exactly **one partition**.

One Partition, One Task

1 A Stage covering 200 partitions → Spark creates **200 Tasks**



2 Each Task ships to an Executor as serialized bytecode + a partition reference



3 Tasks in a Stage run **in parallel** across every available Executor core



4 The Stage only completes once **every** Task inside it finishes

Interview Tip: More partitions ≠ always faster. Too many small Tasks add scheduling overhead; too few limits parallelism.

E FROM CODE TO TASKS

```
df.filter(col("amt") > 0) \  
  .groupBy("city").count() \  
  .show()
```

Tracing the Execution

1 Action (.show()) → 1 Job



DAGScheduler splits at .groupBy() (shuffle) → 2 Stages



Stage 1 — filter (pre-shuffle) → N Tasks, one per input partition



Stage 2 — aggregation (post-shuffle) → M Tasks, one per shuffle partition

Interview Tip: `spark.sql.shuffle.partitions` controls M — the default is 200, and it sets the Task count of every post-shuffle Stage.

Save this slide before your next Data Engineering interview

CONCEPT	TRIGGERED BY	KEY INTERVIEW FACT
Job	One Action	Created once per Action call
Stage	Wide dependency	Boundary = data movement across partitions
Task	One partition	Smallest scheduled unit of work
DAG	Full lineage	Built by DAGScheduler from the logical plan
Shuffle Partitions	spark.sql.shuffle.partitions	Default 200, sets post-shuffle Task count

Follow for daily Data Engineering content

@chintapradeep

Spark · Python · SQL · Interview Prep